

Access Specifiers in C++

C++ has 3 access specifiers:

```
public
private
protected
```

They answer one question:

"Who is allowed to access this member?"

1. public

A **public** member can be accessed **from anywhere**.

```
class Animal {
public:
    int age;
};

int main() {
    Animal a;
    a.age = 10; // ✓
}
```

Think:

public → Everyone

2. private

A **private** member can only be accessed **inside the class that declares it**.

```
class Animal {
private:
    int age;

public:
    void setAge(int x) {
        age = x; // ✓
    }
};

int main() {
    Animal a;
    // a.age = 10; // ✗
}
```

Think:

private → Only the **class** itself

Important:

A derived class **cannot directly access private members** of its base class.

```
class Animal {
private:
    int age;
};

class Dog : public Animal {
    void test() {
        // age = 10; ✗
    }
};
```

The member exists as part of the base-class subobject, but **Dog** cannot directly access it.

3. protected

protected is the interesting one.

It can be accessed:

- 1. Inside the class itself
- 2. Inside derived classes

But **not directly from outside**.

```
class Animal {
protected:
    int age;
};

class Dog : public Animal {
public:
    void setAge() {
        age = 10;    // ✓
    }
};

int main() {
    Dog d;

    // d.age = 10;   ✗
}
```

Think:

```
protected → Class + Derived classes
```

4. The most important table

Memorize this:

AccessSame classDerived classOutsidepublic✓✓✓protected✓✓✗private✓✗✗

This is the **first table you should remember**.

5. Now combine this with inheritance

This is where interview questions usually get interesting.

Consider:

```
class Animal {
public:
    int a;

protected:
    int b;

private:
    int c;
};
```

Now suppose:

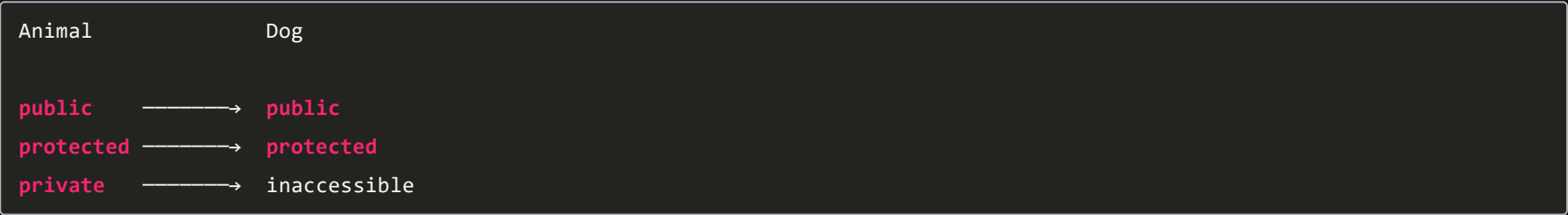
```
class Dog : public Animal {
};
```

What happens?

Public inheritance

Base memberBecomes in Dogpublicpublicprotectedprotectedprivateinaccessible

So:



6. Private inheritance

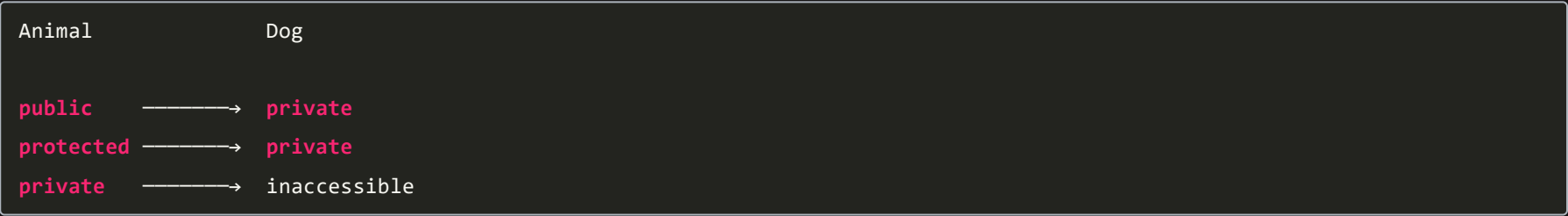
Now:

```
class Dog : private Animal {  
};
```

The access changes:

Base memberBecomes in Dogpublicprivateprotectedprivateprivateinaccessible

So:



7. Protected inheritance

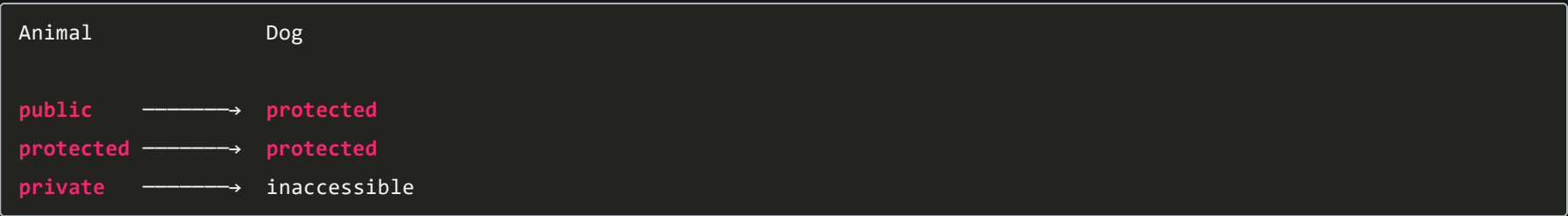
There is also:

```
class Dog : protected Animal {  
};
```

Then:

Base memberBecomes in Dogpublicprotectedprotectedprotectedprotectedprivateinaccessible

So:



8. The master table ★

This is the table I'd memorize for interviews:

Base memberpublicinheritanceprotectedinheritanceprivateinheritancepublicpublicprotectedprivateprotectedprotectedprotectedprotectedprotectedprotectedprotectedprivateprivateinaccessibleinaccessibleinaccessible

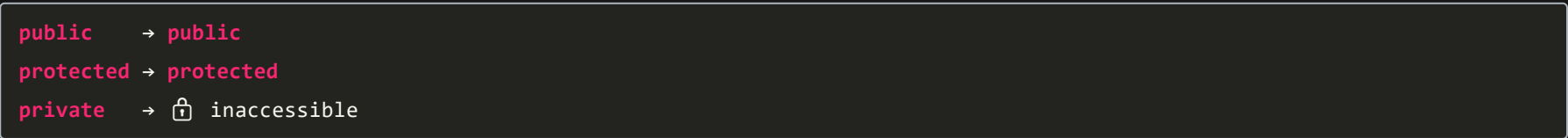
Notice something important:

Base private members are never directly accessible through inheritance.

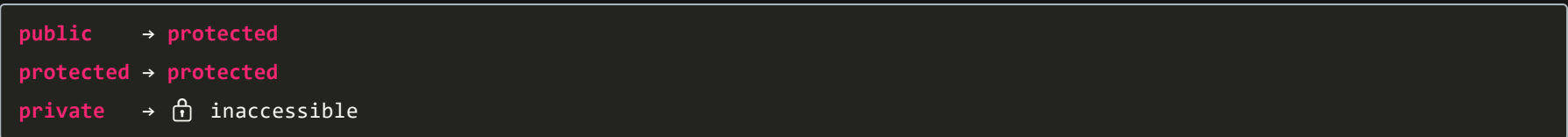
9. A very useful mental model

Think of inheritance access as a visibility transformation.

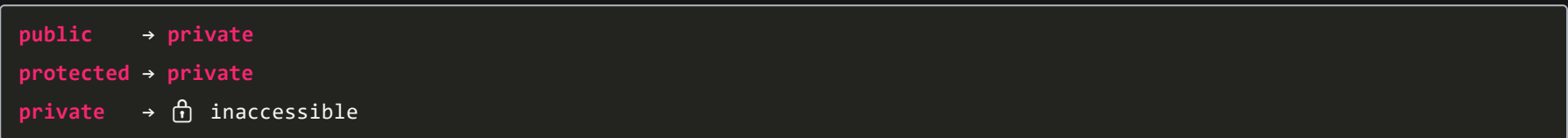
Public inheritance



Protected inheritance



Private inheritance



10. But there's a subtle point about private

Suppose:

```
class Animal {
private:
    int age;
};
```

Then:

```
class Dog : public Animal {
};
```

Does `Dog` contain `age`?

Conceptually, the **base-class subobject contains its own private state**, but `Dog` cannot directly name/access `age`.

So this is invalid:

```
class Dog : public Animal {
public:
    void test() {
        // age = 10; ✗
    }
};
```

But `Animal` can still manipulate its own `age`:

```
class Animal {
private:
    int age;

public:
    void setAge(int x) {
        age = x;
    }
};
```

Then `Dog` can use the public interface:

```
class Dog : public Animal {
public:
    void test() {
        setAge(10); // ☑
    }
};
```

This is a major reason encapsulation works.

11. Why have `protected`?

You might ask:

```
"If private is safer, why do we need protected?"
```

Because sometimes you intentionally want derived classes to access internal state.

Example:

```
class Animal {
protected:
    int age;

public:
    void setAge(int x) {
        age = x;
    }
};

class Dog : public Animal {
public:
    void birthday() {
        age++; // ☑
    }
};
```

But:

```
Dog d;
// d.age++; // ✗
```

So `protected` creates a middle ground:

```
private
  ↓
Only class

protected
  ↓
Class + derived classes

public
  ↓
Everyone
```

12. Interview trap: `protected` doesn't mean "derived classes only"

A common answer is:

"`protected` means only derived classes can access it."

✗ Not completely correct.

The **base class itself** can also access it.

Correct:

`protected` members are accessible inside the declaring class and its derived classes, but not through ordinary outside access.

13. Interview trap: inheritance access vs member access

These are two separate concepts:

Member access

```
class Animal {
public:
    int x;
};
```

Here `public` controls **who can access** `x`.

Inheritance access

```
class Dog : private Animal {
};
```

Here `private` controls **how** `Animal`'s accessible members are exposed through `Dog`.

That's why you can have:

```
class Dog : public Animal
```

and:

```
class Dog : private Animal
```

They don't mean that `Animal` itself is public/private.

14. Why public inheritance is used most often

Usually, when you model:

Dog **IS-A** Animal

you use:

```
class Dog : public Animal
```

because you want:

```
Animal* a = new Dog();
```

to be valid and for the public interface of `Animal` to remain publicly usable through `Dog`.

This is closely related to **polymorphism and the Liskov Substitution Principle**, which you'll encounter later.

Access specifiers

public:		
class		☑
derived		☑
outside		☑
protected:		
class		☑
derived		☑
outside		✗
private:		
class		☑
derived		✗
outside		✗

Inheritance

	public	protected	private
Base public	public	protected	private
Base protected	protected	protected	private
Base private	✗	✗	✗

One sentence to remember

Access specifiers decide who can access a member; inheritance access decides how the base class's public/protected members are exposed in the derived class.

For interviews, master the two tables above. They cover a huge percentage of access-control questions in C++.